

A Additional benchmark details

A.1 Brain-to-Text 2024

B2T ’24 is partitioned into a training set of 8,800 trials, a validation set of 880 trials, and a test set of 1,200 trials (for which ground-truth sentence labels are withheld). Trials are organized into blocks, with each block containing 20–50 sentences. Specifically, the validation split consists of the final block from 22 out of the 24 total recording days. The test split comprises the first two blocks from 15 of the 24 recording days. The training split contains all remaining blocks across the full 24-day recording period. All sentences in this benchmark are sourced exclusively from the Switchboard corpus. All data is publicly available on dryad, with preprocessing details provided in Willett et al. [7].

A.2 Brain-to-Text 2025

B2T ’25 is partitioned into a training set of 8,072 trials, a validation set of 1,426 trials, and a test set of 1,450 trials (for which ground-truth sentence labels are withheld). Trials are grouped into blocks of 5–50 sentences. The validation and test sets combined comprise 67 blocks collected across 41 of the 45 recording days, while the training set comprises 198 blocks spanning all 45 days. The distribution of corpus sources across these splits is detailed in Table A1. The linguistic material is sourced from a diverse set of corpora, including Switchboard, OpenWebText2, Harvard Sentences, a custom high-frequency word list, and random word sequences. All data is publicly available on dryad, with preprocessing details provided in Card et al. [1]

Table A1: Distribution of data sources across training and evaluation splits.

Source	Blocks	Sentences	Split
Switchboard	182	7,672	Train, Val/Test
Freq. Words	48	2,050	Train, Val/Test
50-word Vocab	25	1,091	Train Only
Openwebtext	5	172	Val/Test only
Harvard	3	98	Val/Test only
Random	2	79	Val/Test only

B CTC encoder hyperparameters

For the GRU encoders, we follow the exact same implementation of Willett et al. [7] for B2T ’24 and Card et al. [1] for B2T ’25. For the Transformer encoders, we adopted the parameters from Feghhi et al. [4] for B2T ’24. The hyperparameters for the B2T ’25 time-masked Transformer are listed in Table A2.

C Bounded KV Cache

While GRU architectures compress the input into a fixed hidden state, Transformers require access to all previous input, resulting in an unbounded Key-Value (KV) cache. This is problematic when deploying the speech neuroprostheses in an unstructured, conversational setting where the context is not reset after each sentence. Inspired by Zheng et al.[8], we limit the left context of the causal Time-Masked Transformer to guarantee a constant Key-Value (KV) cache, making the Transformer suitable for real-world, clinical applications.

Table A2: Transformer architecture and training hyperparameters for B2T ’25. Feed-forward network (FFN) multiplier indicates the increase in dimensionality in the FFN module.

Hyperparameter	Value
Patch Length	4 (0.08 ms)
Patch Width	512
Transformer Dimension	384
Layers	5
Heads	6
Head Dimension	64
FFN Multiplier	4
Learning Rate	0.002
White Noise	0.2
Baseline Shift	0.05
Number of Time-Masks	20
Max Mask Length	5
Epochs	500
Dropout	0.35
Input Dropout	0.2
L2 Decay	1×10^{-5}
Gaussian Smooth Kernel Size	20
Gaussian Smooth Width	2.0
Gradient Norm Clipping	100

C.1 Dynamic left temporal context constraint

We define the input sequence as $X = \{x_1, x_2, \dots, x_T\}$, where T is the total sequence length. In a standard causal Transformer, the attention at time step t is computed over all x_i where $i \leq t$. Our model modifies the attention mask $M \in \{0, 1\}^{T \times T}$ such that:

$$M_{t,i} = \begin{cases} 1, & \text{if } \max(0, t - \tau) \leq i \leq t \\ 0, & \text{otherwise} \end{cases}$$

where τ represents the temporal lookback window. During training, τ is sampled stochastically from a uniform distribution $\tau \sim \mathcal{U}(\tau_{min}, \tau_{max})$, where $\tau_{min} = 1\text{s}$ and $\tau_{max} = 20\text{s}$. By dynamically sampling τ during training, the Transformer can operate with a variety of left context sizes during inference.

C.2 KV cache space complexity

Let d be the model dimension and L the number of layers. In a standard causal Transformer, the memory requirement S at time t grows linearly with respect to time for self-attention layers:

$$S_{standard}(t) = t \cdot d \cdot L \cdot 2$$

When restricting left context, the memory requirement is bounded by the maximum context window τ_{max} . Let f_s be the sampling frequency of the encoder (which is the inverse of the encoder frame length). The maximum number of frames stored, N_{max} , is defined as $N_{max} = \tau_{max} \cdot f_s$. The space complexity per inference step is:

$$S_{constrained}(t) = \min(t, N_{max}) \cdot d \cdot L \cdot 2$$

As $t \rightarrow \infty$ (streaming scenarios), the memory footprint remains $O(1)$ relative to the total sequence length T , whereas the standard self-attention computation remains $O(T)$.

C.3 Computational efficiency and stability

Beyond memory, this constraint also ensures constant-time inference per token. In global attention, the t -th token requires $O(t)$ FLOPs for the attention dot-product, leading to a total quadratic complexity of $O(T^2)$. By enforcing the left-context limit, the per-step complexity is capped:

$$\text{Attention FLOPs per step} \approx O(N_{max} \cdot d)$$

This results in a total complexity of $O(T \cdot N_{max})$, which is strictly linear with respect to the input duration.

C.4 Performance

In our experiment, we examined choices of different left context size and their impact on encoder performance measured in Phoneme Error Rate (PER). We show in Figure A1 that encoder perfor-

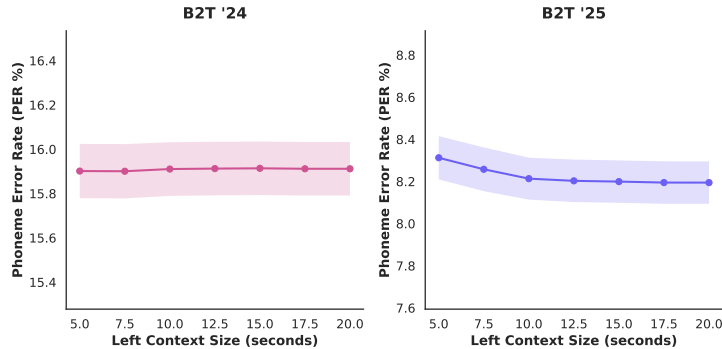


Figure A1: Phoneme Error Rate (PER) as a function of left-context window size for B2T'24 and B2T '25 on validation set.

mance across left-context window sizes remains stable. We used a left context size of 7.5 seconds for B2T '24 and 20 seconds for B2T '25 throughout the paper.

D WFST decoder hyperparameters and additional details

Hyperparameters for the WFST decoder are listed in Table A3 and A4. For additional details on these hyperparameters and additional implementational detail refer to Willett et al. [7].

Table A3: WFST decoding hyperparameters for B2T '24 and '25 benchmarks during first-pass rescoring. These hyperparameters were tuned by the original authors using the baseline GRU encoder, and the same set of hyperparameters were used for analyses with the time-masked Transformer. Variable notation is copied from original authors ([7, 1]).

Hyperparameter	B2T' 24	B2T' 25
Blank Penalty	$\log(7)$	$\log(90)$
Acoustic Scale (α)	0.5	0.325
Beam	18	17
Max Active States	7000	7000
Min Active States	200	200

Table A4: WFST decoding hyperparameters for B2T’24 and ’25 benchmarks during second-pass rescoring. These hyperparameters were tuned by the original authors using the baseline GRU encoder, and the same set of hyperparameters were used for analyses with the time-masked Transformer. Variable notation is copied from original authors ([7, 1]).

Hyperparameter	B2T’ 24	B2T’ 25
Acoustic Scale (α)	0.5	0.325
N-gram LM weight (β)	0.5	0.45
LLM weight	0.5	0.55
N-best beam size	100	100
Used quantized LLM	Yes	No

E LightBeam hyperparameters and additional details

The full list of hyperparameters for *LightBeam* are listed in Table A5. Additional technical details are provided below.

Table A5: *LightBeam* decoding hyperparameters for B2T ’24 and B2T ’25. These hyperparameters were tuned using the GRU encoder, and the same set of hyperparameters were used for analyses with the time-masked Transformer.

Hyperparameter	B2T’ 24	B2T’ 25
LLM rescore interval (Encoder Frames)	10	15
LLM Weight (ϕ)	1.2	1.2
N-gram LM Weight (ω)	0.8	1.0
Acoustic Scale (α)	0.6	0.4
Beam Size (k)	1000	900
Beam Prune Threshold (θ)	22.0	18.0
Homophone Beam Number (h)	3	3
Homophone Prune Threshold (λ)	4.0	4.0
Token Insertion Bonus (β)	1.5	1.5
Word Boundary Bonus (γ)	1.0	1.0
LLM Scoring Chunk Size	256	256

E.1 Beams object

The beam search state is encapsulated in a *Beams* object whose tensors are pre-allocated on the GPU before decoding begins. The object maintains, for each of the K active hypotheses, a cumulative log-probability score $\mathbf{s} \in \mathbb{R}^K$, the most recently emitted non-blank token $\ell_{\text{last}} \in \mathbb{Z}^K$, and a rolling hash $\mathbf{h} \in \mathbb{Z}^K$ of the decoded token sequence. Rather than storing K independent token sequences of length up to T , hypotheses are stored as an implicit tree: a label buffer $\mathbf{W} \in \mathbb{Z}^{K \times T}$ records the token emitted at each step, and a companion pointer buffer $\mathbf{P} \in \mathbb{Z}^{K \times T}$ records which parent beam each hypothesis branched from, allowing the full sequence of any hypothesis to be reconstructed by following pointers in reverse. After each decoding step, hypothesis recombination merges beams that have converged to the same decoded text — detected in $O(K)$ time via hash comparison rather than $O(K^2)$ pairwise string equality — with duplicate scores consolidated via a `max` reduction. Because all buffers are pre-allocated and reused across frames, no dynamic GPU memory allocation occurs during the decoding loop. While the current work decodes single utterances (K beams), the data structure natively supports larger batch sizes by extending all tensors along a batch dimension, enabling straightforward scaling to higher-throughput settings.

E.2 Vectorized Lexicon

Enforcing a lexicon constraint during CTC beam search requires, at each decoding step, determining the set of valid next tokens for every active beam hypothesis. A natural implementation represents the lexicon as a prefix tree (trie), where each node encodes the valid continuations of a phoneme prefix. Given a beam of k hypotheses, the trie-based approach must traverse each hypothesis path independently on the CPU, performing $O(l)$ pointer-chasing dictionary lookups per hypothesis for a prefix of length l , yielding a total cost of $O(kl)$ sequential operations per decoding step. Beyond raw complexity, this approach introduces a critical synchronization bottleneck: the constraint mask must be assembled on the CPU and transferred to the GPU before each log-probability masking step, stalling the GPU pipeline.

To eliminate this bottleneck, we flatten the trie into a dense *transition table* $\mathbf{T} \in \mathbb{Z}^{S \times V}$, where S is the number of trie nodes and V is the vocabulary size. The entry $\mathbf{T}[s, v]$ stores the successor state reached from prefix state s upon observing token v , or a designated sink state s_ϕ for invalid transitions. The prefix state is represented as a single integer tensor $\mathbf{s} \in \mathbb{Z}^K$ residing on the GPU. At each step, computing the constraint mask reduces to a single gather operation, $\mathbf{M} = (\mathbf{T}[\mathbf{s}] \neq s_\phi) \in \{0, 1\}^{K \times V}$, which executes across all K hypotheses simultaneously. Two special cases follow directly from CTC semantics: the blank token is always unmasked, and the most recently emitted non-blank token is always permitted as a repeat. In both cases the prefix state $\mathbf{s}$ is left unchanged, since neither blanks nor repeated tokens advance the decoded prefix. Advancing the prefix state is therefore guarded by $\ell \neq \text{blank}$ and $\ell \neq \ell_{\text{prev}}$, after which the update reduces to a single vectorized lookup $\mathbf{s}' = \mathbf{T}[\mathbf{s}, \ell]$. By keeping all constraint state and computation resident on the GPU, the vectorized lexicon constraint removes the CPU–GPU synchronization overhead entirely and scales gracefully with larger beam widths.

E.3 Homophone tracking and shallow fusion with N-gram LM

Because phoneme-based CTC decoding is agnostic to orthography, a single acoustic beam hypothesis may correspond to multiple valid word spellings (homophones, e.g., *aunt* / *ant*). To handle this, each of the K acoustic beams maintains o orthographic sub-beams — candidate word-level text hypotheses that share the same underlying phoneme path. Each orthographic sub-beam tracks a cumulative N-gram LM score, an integer state ID into the N-Gram LM state registry, and a backpointer into a shared `WordHistory` trie that reconstructs full word sequences without redundant string allocation. Word-level scoring is triggered precisely when a beam emits the word boundary token: the vectorized lexicon state is queried to retrieve all lexicon words that complete at the current prefix, and the N-gram LM is evaluated for each (orthographic context, candidate word) pair. Transition scores are cached by (state ID, word) key so that each unique context–word pair calls the N-gram LM at most once across the entire beam and across utterances. The weighted LM score $\omega \cdot \log P_{\mathcal{G}}(w \mid \text{context})$ is accumulated into each orthographic sub-beam’s running total, and the acoustic beam score is updated by the delta between the new best orthographic score and the previous best.

After scoring, orthographic sub-beams are ranked and the top o candidates are retained. Any candidate whose cumulative LM score falls more than λ log-probability points below the best surviving candidate is discarded, preventing low-probability homophones from inflating memory and compute. The acoustic beam score therefore always reflects the best available word-level interpretation, while the full ranked list of orthographic sub-beams is preserved for subsequent neural LM rescore. The N-gram LM is loaded and queried via Ken-LM [5].

E.4 Delayed fusion with LLM

While the N-gram LM provides efficient word-boundary scoring during the decoding loop, an LLM is additionally used to rescore the o orthographic sub-beams of each acoustic hypothesis at regular intervals during decoding and once more at end-of-sentence. The two models play complementary roles: the N-gram LM is well-suited to the word-boundary expansion step, where the number of orthographic candidates is variable and potentially large due to homophones — its $O(1)$ cached

state transitions make it cheap to score every candidate word in every context. The LLM, by contrast, requires a full forward pass per hypothesis and is therefore best applied when the number of orthographic sub-beams is fixed at o , amortizing its cost over a known, bounded set of candidates. At each rescoreing event, the full word sequences of all orthographic sub-beams across all k acoustic beams are collected and deduplicated, so that identical hypotheses appearing in multiple beams are scored only once. Each unique text is scored by computing $\sum_t \log p_{\text{LLM}}(w_t \mid w_{<t})$, and the resulting score replaces the N-gram score for each orthographic sub-beam. The beam score is then updated by the delta between the new best and old best orthographic sub-beam score.

At the end of decoding, an end-of-sentence scoring pass appends each of the candidate punctuation tokens $.$, $?$, $!$ to every orthographic sub-beam, scores each variant through the LLM, and selects the punctuation that maximizes $\log p_{\text{LLM}}$. This allows the system to jointly infer sentence-final punctuation and refine hypothesis ranking based on full-sequence LLM probability, rather than committing to punctuation as a post-processing step. The result is broadcast back to all requesting sub-beams, ensuring the number of LLM forward passes scales with the number of *unique* hypotheses rather than $k \times o$.

F Next-word prediction finetuning

Next-word prediction finetuning is performed using the training sentences across both datasets. Each sentence is converted to sentence-case because the N-gram LM is not cased. LoRA adapters are applied to all attention projection layers (W_q, W_k, W_v, W_o) and MLP layers with rank $r = 16$ and $\alpha = 32$ [6]. The model is trained for 1 epoch using the AdamW optimizer with learning rate 2×10^{-4} and weight decay 10^{-3} , with a batch size of 16 in BFloat16 precision. Hyperparameters were tuned using the validation sentences from both datasets, and are listed in Table A6.

Table A6: Fine-tuning hyperparameters for the next-word prediction task

Hyperparameter	Value
Base Precision	BFloat16
Optimizer	AdamW
Learning Rate	2×10^{-4}
Weight Decay	10^{-3}
Batch Size	16
Epochs	3
Eval. Frequency	0.25 Epoch
<i>LoRA Configuration</i>	
Rank (r)	16
Alpha (α)	32
Target Modules	W_q, W_k, W_v, W_o

G Generative error correction

We adapt the procedure from Fegghi et al. [4] for generative error correction. To perform LLM fine-tuning, we first generated decoded transcripts using 10 seeds of the time-masked Transformer using the WFST decoder and *Lightbeam* separately. Decoded transcripts were generated by taking the top-scoring beam from each seed across the training, validation, and testing sets. We then fine-tuned Llama 3.1 8B to predict the ground-truth transcript given the 10 decoded sentences for each trial. Fine-tuning was performed by “masking” the ground-truth transcript and training the LLM to predict the masked tokens with cross-entropy loss. The LLM was fine-tuned with QLoRA [3] via the Unsloth package [2]. We used the default hyperparameters provided by Unsloth for Llama 3.1 8B, which are listed in Table A7. A prompt, also provided below, was used to guide the LLM for

each trial. As no hyperparameter tuning was performed, our procedure consisted of fine-tuning the LLM first on the validation set, then on the training set, and finally once more on the validation set. The validation and training sets were combined across B2T '24 and '25. We report results across 10 seeds of fine-tuning.

Table A7: Fine-tuning hyperparameters for the generative error correction

Hyperparameter	Value
Quantization	4-bit NF4 (bitsandbytes)
Optimizer	AdamW-8bit
Learning Rate	2×10^{-4}
Batch Size	16
Epochs	1
Data Scheduling	[val, train, val]
Acceleration Framework	Unsloth
<i>LoRA Configuration</i>	
Rank (r)	16
Alpha (α)	16
Target Modules	W_q, W_k, W_v, W_o , MLP

Your task is to perform automatic speech recognition error correction. Below are multiple candidate transcriptions of the same utterance. These candidates were decoded from neural activity and may contain errors. Based on the candidates, produce the single most accurate, coherent, and grammatical transcription of the original utterance. Focus on key differences between candidates that change meaning or correctness, and avoid repetitive or nonsensical phrases. Respond with only the final corrected transcription--no explanations or extra text.

H Real-Time Factor (RTF)

Real-Time Factor (RTF) is a common metric for measuring the latency of sequence processing systems. Given a system which takes time $f(l)$ to process an input sequence of duration l , the RTF is defined as:

$$\text{RTF} = f(l)/l$$

Decoding with *LightBeam* achieves $\text{RTF} < 1.00$ regardless of encoder model choice on the test set, which makes the decoding pipeline ideal for clinical application (Figure A2).

I Multiple pronunciation variants

We constructed a pronunciation lexicon from the 100k-word vocabulary of the 4-gram language model provided by ImagineVille. Each word was paired with its default CMU pronouncing dictionary phoneme sequence, and words with multiple CMU pronunciations received additional entries in the lexicon (denoted word(2), word(3), etc.) for each distinct variant. Trailing numbers were stripped from words before applying the N-gram LM for scoring. We found that allowing for alternative pronunciations of the same word did not significantly improve performance over just using the default pronunciation for each word.

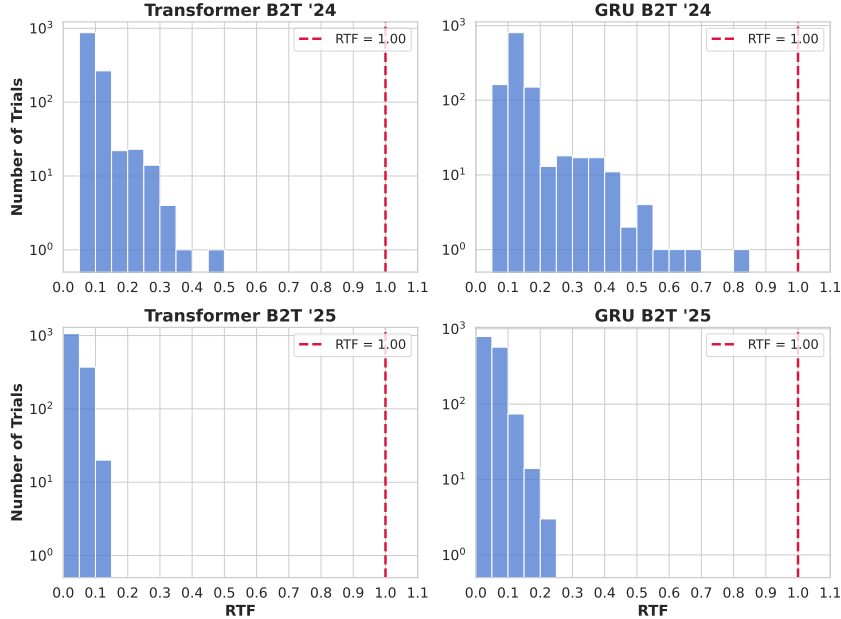


Figure A2: RTF Distributions of *LightBeam* across all trials on the test set of B2T '24 and B2T '25. The scale of the y-axis is logarithmic.

J Llama 3.2 3B Base for delayed fusion

We experimented using a finetuned Llama-3.2-3B in place of the finetuned Llama-3.2-1B for delayed fusion on the validation set for the baseline GRU. We observed statistically significant improvement in performance on B2T '24, but no significant improvement for B2T '25, at the cost of higher RTF and VRAM usage (Table A8).

Table A8: Word Error Rate (%) comparison on the B2T '24 and B2T '25 Validation set when using GRU encoder and different LLMs for Delayed Fusion. Mean $\pm$ SEM over $n = 10$ seeds. († sig. better than 1B Model).

LLM	B2T '24			B2T '25		
	WER	RTF	VRAM	WER	RTF	VRAM
llama-3.2-1B	14.17 $\pm$.22	0.18	6.14	6.36 $\pm$.14	0.06	5.60
llama-3.2-3B	13.82†$\pm$.13	0.26	9.82	6.24 $\pm$.16	0.08	9.64

K Hyperparameter tuning

All hyperparameter tuning was done using the validation sets. For the time-masked Transformer on B2T '25, we performed tuning by starting with the hyperparameters of the B2T '24 model and modifying them manually based on validation phoneme error rate. For *LightBeam*, we performed all hyperparameter tuning manually using the baseline GRU.

References

- [1] Nicholas S Card, Maitreyee Wairagkar, Carrina Iacobacci, Xianda Hou, Tyler Singer-Clark, Francis R Willett, Erin M Kunz, Chaofei Fan, Maryam Vahdati Nia, Darrel R Deo, Aparna Srinivasan, Eun Young Choi, Matthew F Glasser, Leigh R Hochberg, Jaimie M Henderson, Kiarash Shahlaie, Sergey D Stavisky, and David M Brandman. An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7):609–618, August 2024.
- [2] Michael Han Daniel Han and Unsloth team. Unsloth, 2023.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv [cs.LG]*, May 2023.
- [4] Ebrahim Feghhi, Shreyas Kaasyap, Nima Hadidi, and Jonathan Kao. Time-Masked Transformers with Lightweight Test-Time Adaptation for Neural Speech Decoding. In *Proc. Neural Information Processing Systems (NeurIPS)*, page to appear, 2025.
- [5] Kenneth Heafield. KenLM: Faster and smaller language model queries. In *Proceedings of the Sixth Workshop on Statistical Machine Translation*, pages 187–197. Association for Computational Linguistics, 2011.
- [6] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [7] Francis R Willett, Erin M Kunz, Chaofei Fan, Donald T Avansino, Guy H Wilson, Eun Young Choi, Foram Kamdar, Matthew F Glasser, Leigh R Hochberg, Shaul Druckmann, Krishna V Shenoy, and Jaimie M Henderson. A high-performance speech neuroprosthesis. *Nature*, 620(7976):1031–1036, August 2023.
- [8] Kun Zheng, Omer Cengiz, Navdeep Singh, Yuan Shangguan, Jialiang Pan, Xiang Liu, Jayant Kalasapur, Christian Fuegen, Kaustubh Kalgaonkar, and Michael L Seltzer. Dynamic chunk convolution for unified streaming and non-streaming conformer asr. *arXiv preprint arXiv:2304.09325*, April 2023. 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).